



OptiEDT

Automatic University Timetable Generation, Ranking and Explanation

OptiEDT automatically creates valid university timetables, compares different solutions, and helps explain the results.

Author

Mohamed Jawad Tourir

Live demo

<https://optiedt-ai-timetabling.vercel.app>

Company internship project — academic year 2025–2026

THE PROBLEM

Building a timetable by hand does not scale

Many teachers + Many groups + Many rooms + Many time slots + Many rules
↓
Conflicts, and weeks of manual work

A department normally builds its weekly timetable in a spreadsheet. There is no single person who can hold every rule in their head at once — a teacher cannot be in two rooms simultaneously, a room cannot host two sessions at once, a group cannot attend two lectures at the same hour, and every room needs the right type and enough seats.

218	44	51	20	425	30
sessions	teachers	groups	rooms	students	slots/week

This is the reference department OptiEDT was built for. Doing this by hand takes two to three weeks, depends on one experienced person, and conflicts usually surface only after the timetable is already published. The number of possible arrangements is far beyond what anyone can search by hand — this is exactly the kind of problem constraint solving was built for.

THE PIPELINE

What OptiEDT does

Data

- | | |
|----------------|---|
| → Check | five arithmetic checks: is this even possible? |
| → Optimize | CP-SAT places all 218 sessions under 12 unbreakable rules |
| → 3 Candidates | three different priority weightings, three valid timetables |
| → Score & Rank | each scored 0-100 on seven quality measures |
| → Compare | the officer sees exactly why one outranks another |
| → Publish | the department adopts one |
| → Student view | the group's own week, nothing else |

Stages 1 and 2 always run. A third stage — diagnosis — runs only if no valid timetable exists at all: instead of a bare failure, it reports the smallest set of conflicting rules, so the officer knows exactly what to change.

Solving takes a few minutes on capable hardware, so the browser never sits waiting for it — a request returns immediately with a reference number, and the screen checks on progress.

THE APPLICATION

Signing in

SCREENSHOT — Sign-in screen

Capture: open the live demo URL, before signing in.

Shows the four role buttons (Timetable Officer, Administrator, Teacher, Student) and the username/password fields.

Four roles share the application. Clicking a role button only fills in the matching username — what an account can actually do comes from the account itself, checked on the server, not from which button was clicked.

ROLE	WHAT THIS PERSON DOES
Timetable Officer	Loads data, generates timetables, compares and publishes them
Administrator	Manages the academic calendar and user accounts
Teacher	Declares availability, reads their own timetable
Student	Reads their group's published timetable

A dark navigation strip on the left groups the screens into Timetabling, Inputs and Institution. Each role sees only the screens relevant to it — a teacher never sees the Generate screen at all.

GENERATION

Generating a timetable

SCREENSHOT — Generate screen, a completed run

Capture: sign in as Timetable Officer, open Generate. Shows the pipeline stages and the three ranked candidates with their scores.

1. The officer chooses a search effort and presses **Generate timetable**.
2. Stage 1 checks the data — is a solution even possible?
3. Stage 2 runs CP-SAT three times, under three different priority weightings.
4. Each result is scored on seven quality measures and ranked.
5. The three candidates appear, best first.

This takes time because CP-SAT is genuinely searching a very large space for a provably valid answer, not approximating one. On the developer's own machine, using every processor core, the reference run above completed in 287 seconds. On this deployment's free hosting tier — a small fraction of a CPU core — the same search can take twenty minutes or more. That is a hosting-plan limit, not a limit of the algorithm.

THE RULES THAT CANNOT BE BROKEN

Hard constraints

A hard constraint is a rule with no exceptions. If a timetable would break one, it is not produced — not scored down, not accepted as "close enough". Twelve such rules govern every timetable OptiEDT builds:

CODE	RULE
H1	A teacher cannot teach two sessions at once
H2	A group cannot attend two sessions at once
H3	A room cannot host two sessions at once
H4	A session must be placed in a room of the right type
H5	A room must be large enough for the group
H6	A teacher's declared unavailability must be respected
H7	Every session gets exactly one time and one room
H8	A multi-hour session stays within a single day
H9	Nothing is placed on a closed slot or holiday
H10	A session the officer has locked stays where it is
H11	Total demand never exceeds total room capacity
H12	If a whole year group is busy, its sub-groups are too

If these twelve rules cannot all be satisfied together, OptiEDT reports exactly which ones conflict, rather than returning an invalid timetable or searching forever.

BEYOND "VALID"

Quality and ranking

A valid timetable is not automatically the best one. Among all the timetables that break none of the twelve rules, some are still better than others — fewer gaps in a student's day, more teachers getting their preferred hours, and so on. OptiEDT generates three valid candidates under three different priorities and scores each one out of 100:

CANDIDATE	PRIORITY	SCORE
Candidate 1	Balanced	81.438
Candidate 2	Student-favouring	80.572
Candidate 3	Teacher-favouring	80.611

The score is a plain weighted sum of seven measures — nothing more elaborate, deliberately. Because it is only a sum, it can be checked with a calculator:

```

S2  1.000000 × 0.25/0.90 = 0.2778
S3  0.981383 × 0.15/0.90 = 0.1636
S4  0.622093 × 0.10/0.90 = 0.0691
S5  0.550459 × 0.20/0.90 = 0.1223
S6  0.861985 × 0.10/0.90 = 0.0958
S7  0.772388 × 0.10/0.90 = 0.0858
S10 0.233333 × 0.00/0.90 = 0.0000
      × 100 = 81.4384272710004

```

That is the exact figure the live system displays for candidate 1 — not an approximation, the same number to the last decimal.

UNDERSTANDING THE RANKING

The Compare screen

SCREENSHOT — Compare screen, decomposition ledger

Capture: sign in as Timetable Officer, open Compare, select two candidates. Shows the two scores side by side and the criterion-by-criterion breakdown.

This screen answers the question a department head will always ask: *why is this one ranked above the other?* Rather than a summary, it shows the actual calculation — each of the seven quality measures contributes a signed amount to the score difference, and every contribution is added up in front of the reader:

CRITERION	CONTRIBUTION
Student idle time (S2)	+1.736
Teacher idle time (S3)	-0.310
Teacher preference (S5)	-0.612
Sum of contributions	+0.827
Difference in score	+0.827

These two numbers are always identical — verified against the live system at zero error. The explanation is not a summary of the calculation; it *is* the calculation, read term by term.

EXPLANATION, NOT DECISION

The AI assistant

SCREENSHOT — the "OptiEDT AI" panel on the Compare screen

Capture: on Compare, with two candidates selected, alongside the decomposition table on the right-hand side. Shows the label "Written by the language service" above an answer.

WHAT IT DOES

- Explains why a candidate scores as it does
- Answers questions about the run's own figures
- Summarises a candidate's strengths and weaknesses
- Suggests a change for a human to approve

WHAT IT NEVER DOES

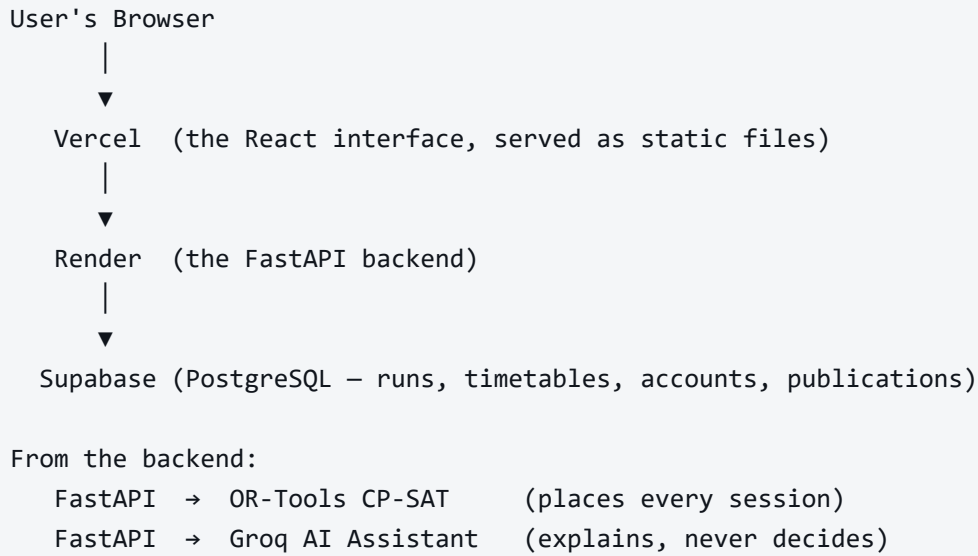
- Generate the timetable
- Change a score or the ranking
- Move or edit a session
- Touch the database

Optimization and AI are kept strictly separate. The optimizer decides what is a valid timetable. The scoring engine decides what is a better one. The AI only reads numbers already produced by those two and writes sentences about them — every figure it states is checked against the numbers it was given, and an answer with an invented figure is discarded entirely rather than shown with one wrong number in it.

Tested directly against the live system: asked a question its supplied data could not answer, the assistant replied that the information was not available — rather than guessing. That refusal is the safeguard working as intended.

HOW IT IS BUILT

Architecture



Every request from the browser goes to Vercel, which forwards anything meant for the backend straight through to Render. The frontend never talks to the database or the solver directly — only the backend does, which is what keeps the guarantees in section 6 enforceable rather than optional.

WHY THESE TOOLS

Technology choices

TECHNOLOGY	WHY IT WAS CHOSEN
React + Vite	Fast, simple frontend that builds to static files anyone can host
FastAPI	Python backend, sitting naturally next to the Python solver
OR-Tools CP-SAT	Purpose-built for exactly this kind of constraint problem
PostgreSQL / Supabase	A reliable, managed relational database
Vercel	Serves the static frontend worldwide for free, redeploys on push
Render	A server that stays alive — the solver needs minutes, not seconds
Groq (AI)	Used only to write plain-language explanations, nothing else

Why not host everything on Vercel?

Vercel runs code in short-lived functions that are stopped after seconds. A CP-SAT solve takes minutes in a background process. Checked directly against Vercel's own documentation before deciding: the function would be killed mid-solve, a second request could land on a different instance and lose the result, and the "one solve at a time" rule the system depends on would break under load. The fix needed one small configuration file and no change to the application itself.

REAL ENGINEERING

Problems and solutions

1. A reassuring number was the wrong measure

Discovered: overall week occupancy read as 71% — comfortable. **Investigation:** the real risk sits in two-hour windows, which were at 91% with almost no spare capacity. **Result:** the instance was nearly infeasible, and I had trusted the easy number instead of trying to disprove it. Cost several days.

2. The solver's own reported score didn't match its answer

Discovered: the score the solver library reported and the score its own solution actually earned disagreed. **Investigation:** the scoring engine recomputes every score independently from the real placements rather than trusting the solver's internal figure. **Result:** that independence caught a genuine bug in the third-party library — nothing in the product broke.

3. Docker looked successful while reaching the wrong database

Discovered: another PostgreSQL was already running on the same port. **Investigation:** the container reported "healthy" while every connection silently reached the other server. **Result:** the port is now configurable, and the exact server version is checked before trusting a connection.

4. One hard-coded setting blocked deployment entirely

Discovered: the list of websites allowed to call the API was a fixed value in the source code.

Investigation: it worked on the developer's machine and nowhere else. **Result:** made configurable — the one change that unblocked deployment.

5. The obvious hosting choice was the wrong one

Discovered: Vercel seemed like the natural place for the whole application. **Investigation:** its functions are killed after seconds; the solver runs for minutes. **Result:** split the deployment — frontend on Vercel, backend on a server that stays alive — with one small configuration file and no change to the application's own code.

BEING HONEST

Current limitations

These are limits of the **current free-tier demo deployment**, not of the software's design:

LIMITATION	WHY
First load takes up to a minute	The free server sleeps when idle
A new generation run is slow here	Free tier gives a fraction of a CPU core; the same run takes under 5 minutes on ordinary hardware
Examination results aren't saved	By design — they exist only while the server process runs
Demo accounts share one password	Demonstration seeding, not how real accounts work

What I would improve in a future version

- Persist examination results instead of keeping them in memory only
- Move the backend to a tier with a full CPU core, cutting generation time from minutes to seconds
- Individual accounts and passwords instead of a shared demo password

TRY IT YOURSELF

How to test the demo

Demo password (all four accounts): G9AieFTxyTlxqVz1jfD1tQ

Username: responsable (Officer), administrateur (Administrator), t001 (Teacher), etudiant (Student).

This document is private — please don't forward it or post it publicly.

1. Open the live URL. Allow up to a minute for the first load.
2. Sign in as Timetable Officer.
3. Open **Generate** — a completed run with three candidates is already loaded.
4. Inspect the three candidates and their scores.
5. Open **Timetables** and switch between the by-teacher, by-group, by-room and occupancy views.
6. Open **Compare**, pick two candidates, and read the score breakdown.
7. Open the **Published** screen and see the adopted timetable with its full record.
8. Sign out, sign back in as **Student**, and check **My Timetable**.
9. Sign back in as Officer and try **Examinations** — generates in about half a minute.
10. Return to **Compare** and ask the AI assistant a supported question.
11. Sign out.

Supervisor communication. [Screenshot of the WhatsApp message announcing the project documents to be inserted here — see the note at the end of this document.]

SPEAKING ABOUT IT

Questions my supervisor may ask

What is OptiEDT?

A system that automatically builds a university department's weekly timetable, ranks several valid options, and explains the ranking in plain language.

What problem are you solving?

Manual timetabling takes weeks, depends on one person, and conflicts surface only after publication. OptiEDT builds a conflict-free timetable automatically in minutes.

Why is timetable generation difficult?

Hundreds of sessions, teachers, groups and rooms all have to satisfy many rules simultaneously — the number of possible arrangements is far beyond manual search.

Why did you use optimization instead of AI to build the timetable?

Because I needed a guarantee, not a good guess. A constraint solver makes an invalid timetable structurally impossible; a learned model would only make one less likely.

Why CP-SAT / OR-Tools specifically?

It is built exactly for this kind of problem — assigning things under hard rules — and is a mature, well-tested Google library.

What are hard constraints?

Rules that cannot be broken under any circumstance, like a teacher never teaching two sessions at once. Breaking one means the timetable isn't produced at all.

What are soft constraints?

Quality preferences, like minimising gaps in a student's day. They don't invalidate a timetable — they change its score.

Why generate three candidates instead of one?

Because "best" depends on priorities — favouring students, favouring teachers, or balancing both. All three are valid; the choice is a policy decision for a person, not the algorithm.

How is the score calculated?

A plain weighted sum of seven quality measures, each between 0 and 1, multiplied by its weight and summed to 100.

How do you know the score is correct?

Because it's just arithmetic — I recomputed it by hand from the published figures and it matched the system's number to the last digit.

Where is the AI?

Only in the explanations. It reads figures the system already calculated and writes sentences about them.

Can the AI change the timetable?

No. It cannot place a session, change a score, or touch the database — it only produces text, and even that text is checked against the real numbers before being shown.

Why keep optimization and AI separate?

So a mistake in the AI can only produce a wrong sentence, never a wrong timetable or a wrong score. The two failure modes are kept apart on purpose.

Why React/Vite for the frontend?

Fast to build with, and it compiles to plain static files that are cheap and simple to host.

Why FastAPI for the backend?

A Python framework, so it sits naturally beside the Python solver code.

Why PostgreSQL?

A reliable, well-understood relational database for runs, candidates and accounts.

Why Supabase?

Managed PostgreSQL hosting — no server to maintain myself.

Why Vercel?

Free, fast static hosting for the frontend, with automatic redeploys.

Why Render?

It runs a persistent process, which the solver needs — Vercel's functions get killed after seconds.

Why not host everything on Vercel?

A CP-SAT solve takes minutes; Vercel would kill it partway through every time.

What happens when I click Generate?

The system checks the data, then runs the solver three times under different priorities, scores each result, and ranks them.

What happens when a timetable is published?

It becomes visible to teachers and students, along with a full record of exactly which run and settings produced it.

What does a student see?

Only their own group's published timetable — nothing about other groups or unpublished drafts.

What was the hardest technical problem?

Trusting a reassuring occupancy number that was actually measuring the wrong thing, and nearly missing that the instance was close to infeasible.

What would you improve in a future version?

Persisting examination results, running on a server with a full CPU core, and giving every account its own individual password.

READY TO SAY OUT LOUD

My 60-second explanation

"I built OptiEDT to solve a real problem: my university's department builds its timetable by hand in a spreadsheet, and it takes two to three weeks and still produces conflicts. I used a constraint solver called OR-Tools, which places every session — 218 of them — under rules that can never be broken, so the result is guaranteed conflict-free. It doesn't just produce one timetable — it produces three, each with different priorities, and scores them out of 100 so the department can pick the one that fits them best. That score isn't a black box — it's a plain weighted sum, so anyone can check it with a calculator. Then I added an AI assistant, but only to explain the results in plain language — it never decides anything, and if it ever states a number that doesn't match what the system actually calculated, that answer gets thrown away automatically. For deployment, I split it across three services: the interface on Vercel, the backend on Render because the solver needs to run for minutes at a time, and the database on Supabase. It's live right now, and anyone can test it."

My 5-minute explanation

The problem. Manual timetabling for a department this size — 218 sessions, 44 teachers, 51 groups, 20 rooms — takes two to three weeks and conflicts still slip through.

My approach. I split the work into two separate decisions that never mix: what is a *valid* timetable, and what makes one valid timetable *better* than another.

Optimization. A constraint solver, OR-Tools CP-SAT, handles validity. It places every session under twelve rules that cannot be broken. If they can't all be satisfied, it reports exactly which rules conflict instead of guessing.

Ranking. Quality is a separate, much simpler question: a plain weighted sum of seven measures. Because it's just arithmetic, the difference between any two timetables can be broken down term by term and verified by hand — I did exactly that and it matched the system to the last digit.

The AI. It only explains results that already exist. It can't place a session, can't change a score, can't touch the database, and every number it writes is checked against the real figures before being shown.

Architecture. React on Vercel for the interface, FastAPI on Render for the backend and solver, PostgreSQL on Supabase for the data. Vercel couldn't run the solver itself — its functions get killed after seconds, and a solve takes minutes — so I checked that against Vercel's own documentation and split the deployment accordingly.

Problems I solved. I nearly missed that the instance was close to infeasible because I trusted a reassuring

overall-occupancy number instead of the two-hour-window measure that actually mattered. I also caught a real bug in the solver library, because the scoring engine recomputes every score independently rather than trusting the solver's own number.

Where it stands. It's deployed and live right now, with real generated timetables you can inspect, compare and publish today.